

Wordle

LEGAL NAME	<i>LEONARD-ADRIAN BUNEA</i>
PREFERRED NAME	ZOE MCFIFE
TIME SPENT	04:25:21

Table of Contents

Table of Contents	1
Table of Figures.....	2
Exercise 1 – File I/O and Error Handling	3
Solution Idea	3
Code.....	3
Test	4
Exercise 2 – Input Validation.....	6
Solution Idea.....	6
Code.....	6
Test	8
Exercise 3 –Feedback Calculation.....	9
Solution Idea.....	9
Code.....	9
Test	11
Exercise 4 – Autosolve Feature.....	12
Solution Idea.....	12
Code.....	12
Test	13
Exercise 5 – Complete Game Implementation	15

Table of Figures

Figure 1: Read Words	4
Figure 2: Global Words Class.....	4
Figure 3: reading words.....	5
Figure 4: Reading Words	5
Figure 5: Word Reading Failure.....	5
Figure 6: getStringInput Method	7
Figure 7: validateGuess Method.....	7
Figure 8: enterGuess Method	8
Figure 9: Guessing Tests	8
Figure 10: computeGuess	10
Figure 11: Guess.....	11
Figure 12: Wordle	11
Figure 13: finding Hints.....	13
Figure 14: HELP!.....	13
Figure 15: TWANG	14
Figure 16: Main Menu	15
Figure 17: Help	15
Figure 18: Wordle	16

Exercise 1 – File I/O and Error Handling

Solution Idea

One can use a buffered reader to read a CSV file. I also save the words in memory and the guesses in a HashSet so looking for a valid word is more efficient. I exit the program inside the catch clause if the CSV file isn't found or able to be read.

Code

```
/**
 * Reads a CSV file and returns a set of words.
 *
 * @param file The path to the CSV file.
 * @return A set of words read from the file.
 */
public static Set<String> readCsvFile(String file)
{
    Set<String> words = new HashSet<>();

    try (BufferedReader reader =
Files.newBufferedReader(Path.of(file)))
    {
        String line;

        // Skip header
        reader.readLine();

        while ((line = reader.readLine()) != null)
        {
            line = line.trim().toLowerCase();

            if (!line.isEmpty())
            {
                words.add(line);
            }
        }
    }
    catch (IOException e)
    {

```

Wordle

```
        System.err.println("Error reading file: " + file);
        System.err.println(e.getMessage());
        System.err.println("Make sure the file exists and is
accessible. Program will exit.");
        System.exit(1);
    }

    return words;
}
```

Figure 1: Read Words

```
/**
 * Global class to store valid Wordle words.
 */
public class Words
{
    /**
     * List of valid Wordle solution words.
     */
    public static List<String> validSolutions = new
    ArrayList<String>();

    /**
     * Set of valid Wordle guess words. Using a set for O(1)
    lookup time.
     */
    public static Set<String> validGuesses = new HashSet<>();
}
```

Figure 2: Global Words Class

Test

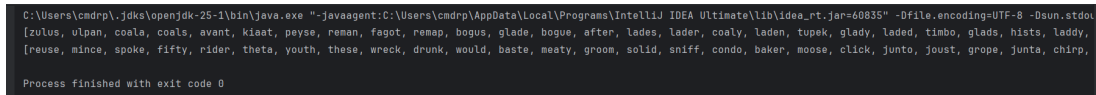
```
public static void main(String[] args)
{
    WordleWordReader.readWordleWords();

    IO.println(Global.Words.validGuesses);
}
```

Wordle

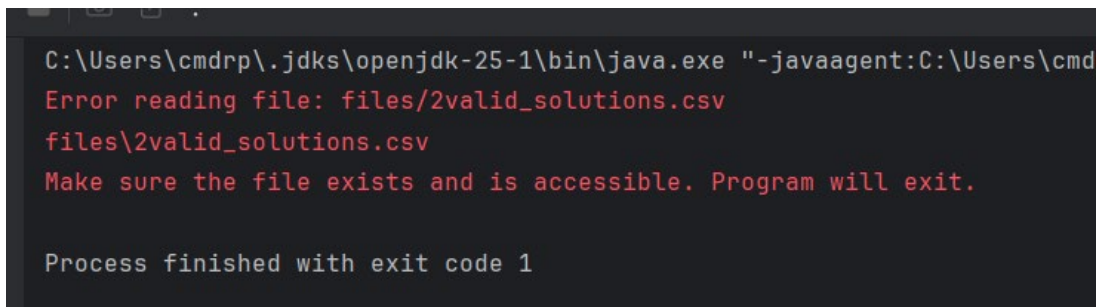
```
IO.println(Global.Words.validSolutions);  
}
```

Figure 3: reading words



A terminal window showing the execution of a Java program. The command is `C:\Users\cmdrp\.jdk\openjdk-25-1\bin\java.exe "-javaagent:C:\Users\cmdrp\AppData\Local\Programs\IntelliJ IDEA Ultimate\lib\idea_rt.jar=60835" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8`. The output lists 100 words: [zulus, ulpan, coala, coals, avant, kiaat, payse, reman, fagot, remap, bogus, glade, bogue, after, lades, lader, coaly, laden, tupek, glady, laded, timbo, glads, histis, laddy, reuse, mince, spoke, fifty, rider, theta, youth, these, wreck, drunk, would, baste, meaty, groom, solid, sniff, condo, baker, moose, click, junto, joust, grope, junta, chirp,]. The process finished with exit code 0.

Figure 4: Reading Words



A terminal window showing an error. The command is `C:\Users\cmdrp\.jdk\openjdk-25-1\bin\java.exe "-javaagent:C:\Users\cmdrp\AppData\Local\Programs\IntelliJ IDEA Ultimate\lib\idea_rt.jar=60835" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8`. The output is: `Error reading file: files/2valid_solutions.csv`, `files\2valid_solutions.csv`, `Make sure the file exists and is accessible. Program will exit.`, and `Process finished with exit code 1`.

Figure 5: Word Reading Failure

Exercise 2 – Input Validation

Solution Idea

First, I created a method that uses `IO.readLine` to read in a `String`, if the `String` is empty, longer than `maxLength` or shorter than `minLength`, it reprompts the user to enter their input. Inside the `enterGuess` method there's a check, that looks up if the entered guess is contained inside the `guess HashSet`.

Code

```
/**
 * Gets a string input from the player with minimum and maximum
 * length restrictions.
 * Continues to prompt until valid input is provided.
 *
 * @param prompt    The prompt message to display
 * @param minLength The minimum allowed length of the input
 * @param maxLength The maximum allowed length of the input
 * @return The validated string input
 */
public static String getStringInput(String prompt, int
minLength, int maxLength)
{
    while (true)
    {
        IO.println(prompt + ": ");
        String input = IO.readLine().trim();

        if (!input.isEmpty() && input.length() <= maxLength &&
input.length() >= minLength)
        {
            return input;
        }

        if (input.length() > maxLength)
        {
            printlnRed("Input exceeds maximum length of " +
maxLength + " characters. Please try again.");
        }
        else if (input.length() < minLength)
```

Wordle

```
{
    printlnRed("Input must be at least " + minLength + "
characters long. Please try again.");
}
else
{
    printlnRed("Input cannot be empty. Please try
again.");
}
}
```

Figure 6: getStringInput Method

```
private boolean validateGuess(String guess)
{
    return Words.validGuesses.contains(guess);
}
```

Figure 7: validateGuess Method

```
private Guess enterGuess()
{
    do
    {
        String currentGuess = UI.getStringInput("Enter your
Guess", Config.MAX_WORD_LENGTH, Config.MAX_WORD_LENGTH
).toUpperCase();

        if (currentGuess.equals(Config.EXIT_COMMAND))
        {
            return new Guess("EXIT!", new GuessAccuracy[0]);
        }
        else if (currentGuess.equals(Config.HELP_COMMAND))
        {
            UI.printlnRed("Not implemented yet.");
        }

        if (validateGuess(currentGuess))
        {
            return computeGuess(currentGuess);
        }
    }
}
```


Wordle

```
        }  
        else  
        {  
            UI.printlnRed("Invalid Guess. Please try again.");  
        }  
    }  
    while (true);  
}
```

Figure 8: enterGuess Method

Test

```
Enter your Guess:  
arf  
Input must be at least 5 characters long. Please try again.  
Enter your Guess:  
this is a long input string that wont work  
Input exceeds maximum length of 5 characters. Please try again.  
Enter your Guess:  
asdf  
Invalid Guess. Please try again.  
Enter your Guess:  
dumbs  
,
```

Figure 9: Guessing Tests

Exercise 3 -Feedback Calculation

Solution Idea

Guesses are stored as pairs of a String and an array of GuessAccuracies, which is an enum with "CORRECT, WRONG_POSITION, INCORRECT" as values. Calculating how accurate the guess is, is done in two passes. In the first pass, it just checks if any letters are already in the correct position and stores those letters. In the second pass it checks for any incorrectly positioned letters. If the letter appears in the word, it adds a WRONG_POSITION GuessAccuracy, otherwise INCORRECT. The GuessAccuracies get displayed as different colors when the guesses are printed to the console.

Code

```
private Guess computeGuess(String guessedWord)
{
    GuessAccuracy[] accuracies = new
    GuessAccuracy[Config.MAX_WORD_LENGTH];
    boolean[] letterUsedInPickedWord = new
    boolean[Config.MAX_WORD_LENGTH];

    // First pass: Check for correct letters in correct
    positions
    for (int i = 0; i < Config.MAX_WORD_LENGTH; i++)
    {
        if (guessedWord.charAt(i) == pickedWord.charAt(i))
        {
            accuracies[i] = GuessAccuracy.CORRECT;
            letterUsedInPickedWord[i] = true;
        }
    }

    // Second pass: Check for correct letters in wrong positions
    for (int i = 0; i < Config.MAX_WORD_LENGTH; i++)
    {
        if (accuracies[i] == null)
        {
            boolean found = false;
            for (int j = 0; j < Config.MAX_WORD_LENGTH; j++)
            {
```

Wordle

```
        if (!letterUsedInPickedWord[j] &&
guessedWord.charAt(i) == pickedWord.charAt(j))
        {
            found = true;
            letterUsedInPickedWord[j] = true;
            break;
        }
    }
    accuracies[i] = found ? GuessAccuracy.WRONG_POSITION
: GuessAccuracy.INCORRECT;
}
}

return new Guess(guessedWord, accuracies);
}
```

Figure 10: computeGuess

```
public class Guess
{
    private final String word;
    private final GuessAccuracy[] accuracies;

    public Guess(String word, GuessAccuracy[] accuracies)
    {
        this.word = word;
        this.accuracies = accuracies;
    }

    public void printGuess()
    {
        for (int i = 0; i < word.length(); i++)
        {
            char letter = word.charAt(i);
            GuessAccuracy accuracy = accuracies[i];

            switch (accuracy)
            {
                case CORRECT ->
UI.printGreen(String.valueOf(letter));
                case WRONG_POSITION ->
```

Wordle

```
UI.printYellow(String.valueOf(letter));
        case INCORRECT ->
UI.printGray(String.valueOf(letter));
    }
    }
    UI.printBlankSeparatorLine();
}

public String getWord()
{
    return word;
}

public GuessAccuracy[] getAccuracies()
{
    return accuracies;
}
}
```

Figure 11: Guess

Test

```
Congratulations! You've guessed the word in 4 attempts!
DUMBS
REALS
PLANE
APPLE
-----
-----
Press Enter to continue...|
```

Figure 12: Wordle

Exercise 4 – Autosolve Feature

Solution Idea

I just look through every word and if a yet unguessed letter is in the correct position of the word, I use that word as the guess. I also shuffle the words list before, so the guesses are more varied.

Code

```
private Guess findHintGuess()
{
    char[] missingLetters = getMissingLetters();

    char[] emptyArray = new char[Config.MAX_WORD_LENGTH];
    for (int i = 0; i < emptyArray.length; i++)
    {
        emptyArray[i] = Config.PLACEHOLDER_LETTER;
    }

    if (Arrays.equals(missingLetters, emptyArray))
    {
        UI.printlnRed("No missing letters to provide hints
for.");
        UI.delayMedium();
        return computeGuess("APPLE");
    }

    UI.printlnYellow(Arrays.toString(missingLetters));
    UI.delayMedium();

    List<String> words = new ArrayList<>();
    words.addAll(Words.validGuesses);
    Collections.shuffle(words);

    for (String word : words)
    {
        int availableHints = Config.HINT_ACCURACY;

        IO.println(word.length() + " " + word);
```

Wordle

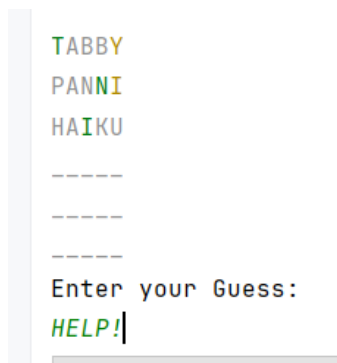
```
for (int i = 0; i < word.length(); i++)
{
    if (missingLetters[i] != Config.PLACEHOLDER_LETTER)
    {
        if (word.charAt(i) == missingLetters[i])
        {
            availableHints--;

            if (availableHints <= 0)
            {
                return computeGuess(word);
            }
        }
    }
}

UI.printlnRed("No more hints available. Entering APPLE as
default guess.");
UI.delayMedium();
return computeGuess("APPLE");
}
```

Figure 13: finding Hints

Test



```
TABBY
PANNI
HAIKU
-----
-----
-----
Enter your Guess:
HELP!|
```

Figure 14: HELP!

Wordle

TABBY

PANNI

HAIKU

TWANG

Enter your Guess:

'

Figure 15: TWANG

Exercise 5 – Complete Game Implementation

```

      --      --      .---.---
    /  \    /  \_____  _|_/_|  |  _---
   \   \/\ /   /  _\  _  \/_  _|  |  |  _/_  \
    \         ( <_> ) | \/_/_/ |  |  |_\  _--/_
      \_/_\ /  \_--/_|_  |  \_--  |  |_--/_\_--  >
          \/\              \/\              \/\

Zoe Edition

Main Menu
-----
1. Start Game
2. Help
3. Quit Game
Select 1 - 3: > |

```

Figure 16: Main Menu

```

===== Help =====
Type HELP! to receive a hint for the current word.
Type EXIT! to exit the current game.
Press Enter to continue...

```

Figure 17: Help

Wordle

APPLE

PLANE

COOLS

MAPLE

Enter your Guess:

Figure 18: Wordle